

Hoja de Trabajo #2: Transformers y Mecanismos de Atención

CC3092 – Deep Learning y Sistemas Inteligentes

Javier España (Carné: 23361) — Universidad del Valle de Guatemala

Enlace al repositorio Git: <https://github.com/Javier-Espana/HT2-DL.git>

5 de octubre de 2026

Resumen

Este informe analiza los fundamentos teóricos y experimentales de la arquitectura Transformer y los mecanismos de atención. Se examina la superación del cuello de botella de los modelos recurrentes (RNN seq2seq) mediante el mecanismo de Bahdanau, la analogía Query-Key-Value y las diferencias entre atención aditiva y multiplicativa. Se presenta la derivación matemática de la varianza en *Scaled Dot-Product Attention* ($\text{Var}(q \cdot k) = d_k$), justificando la necesidad de escalar por $\sqrt{d_k}$ para evitar el desvanecimiento de gradientes en la softmax. Mediante experimentación con BERT y GPT-2, se visualizan mapas de atención multifuncionales, se demuestra la resolución de correferencias en esquemas de Winograd, se cuantifica el fenómeno de sumideros de atención (*attention sinks*) en arquitecturas autorregresivas y se evalúa la evolución de la entropía de atención a lo largo de las capas.

1. De las Redes Recurrentes a los Mecanismos de Atención

Cuello de botella en seq2seq y solución de Bahdanau: En los modelos encoder-decoder recurrentes (RNN/LSTM/GRU), el encoder comprime la secuencia completa en un único vector de contexto estático $h_n \in \mathbb{R}^d$. Para oraciones largas, este vector representa un severo cuello de botella de capacidad informativa: los primeros tokens sufren desvanecimiento progresivo y pérdida irrecuperable de detalle fino debido a la acumulación sucesiva de no linealidades. Bahdanau et al. [1] resolvieron este problema permitiendo que el decoder consulte *todos* los estados ocultos intermedios $\{h_1, \dots, h_n\}$. En cada paso t , se computa una distribución de atención $\alpha_{t,j}$ para formar un vector de contexto dinámico $c_t = \sum_{j=1}^n \alpha_{t,j} h_j$, manteniendo acceso directo a cualquier posición de la fuente sin compresión forzada.

Atención aditiva vs. multiplicativa (Luong): Bahdanau et al. [1] introdujeron la atención aditiva usando una red multicapa: $e_{ij} = v_a^T \tanh(W_a s_{i-1} + U_a h_j)$. Por su parte, Luong et al. [2] propusieron la atención multiplicativa o de producto punto: $e_{ij} = s_i^T W_a h_j$ (o $s_i^T h_j$). En hardware moderno (GPUs/TPUs), la atención multiplicativa es drásticamente más eficiente: se ejecuta como una única multiplicación matricial generalizada (GEMM), explotando al máximo los núcleos tensoriales con rutinas BLAS optimizadas en memoria contigua, mientras que la aditiva requiere asignar tensores intermedios y aplicar $\tanh$ elemento por elemento.

Analogía Query (Q), Key (K) y Value (V): El mecanismo equivale a una búsqueda asociativa continua en un diccionario. En lugar de comparar una consulta con una clave binaria, la consulta q evalúa su similitud con cada llave k_i . Tras normalizar las afinidades con softmax, se obtiene una combinación lineal ponderada de los valores: $\sum_i \alpha_i v_i$.

- **Self-attention:** Q, K, V nacen de la misma secuencia X , proyectada por matrices entrenables: $Q = XW^Q, K = XW^K$ y $V = XW^V$. Cada token contextualiza su representación atendiendo a todos los tokens del enunciado.
- **Cross-attention:** Las consultas Q provienen del decodificador, mientras que las llaves K y valores V se extraen de la salida del codificador, guiando la generación condicionada por la fuente.

Limitaciones superadas respecto a las RNNs: Al prescindir de la recurrencia, el Transformer elimina tres barreras: (1) *Paralelismo masivo*: el procesamiento de la secuencia completa ocurre de forma simultánea en entrenamiento; (2) *Longitud de camino $O(1)$* : cualquier par de tokens interactúa directamente en una capa de atención, frente a las $O(n)$ transiciones secuenciales de una RNN; (3) *Estabilidad de gradiente*: se reduce el desvanecimiento de gradiente gracias a atajos residuales y atención directa.

2. Arquitectura del Transformer

Scaled Dot-Product Attention y derivación analítica de la varianza: La formulación es $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$. Sean $q, k \in \mathbb{R}^{d_k}$ dos vectores con componentes independientes e idénticamente distribuidos con media $\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$ y varianza $\text{Var}(q_i) = \text{Var}(k_i) = 1$. Su producto punto es $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$. Para cada sumando:

$$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0, \quad \text{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 = \text{Var}(q_i) \text{Var}(k_i) = 1. \quad (1)$$

Por la propiedad de aditividad de la varianza para variables aleatorias independientes:

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k. \quad (2)$$

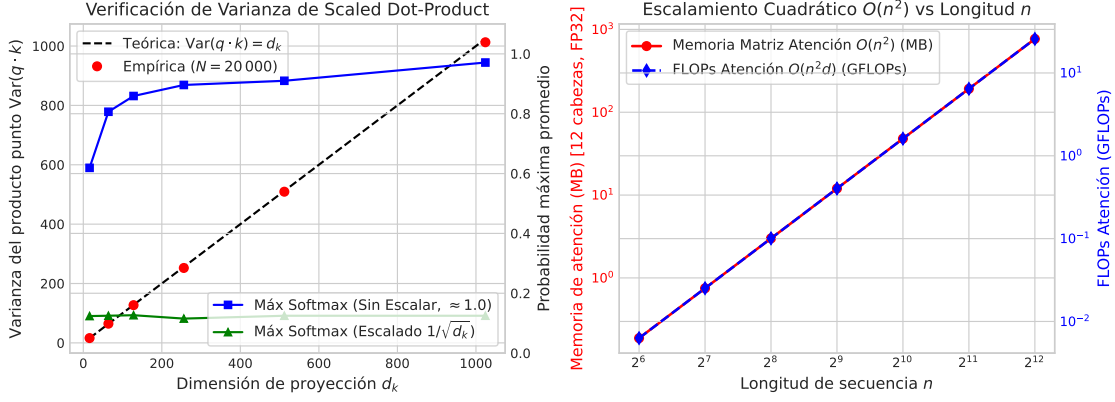


Figura 1: **Izquierda:** Verificación empírica de $\text{Var}(q \cdot k) = d_k$ y saturación del valor máximo de la Softmax cuando se prescinde de $\sqrt{d_k}$. **Derecha:** Escalamiento cuadrático $O(n^2)$ de memoria y FLOPs vs. longitud de secuencia n .

La desviación estándar es $\sqrt{d_k}$. Si se divide entre $\sqrt{d_k}$, la varianza se normaliza a la unidad:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{1}{d_k} \text{Var}(q \cdot k) = \frac{d_k}{d_k} = 1. \quad (3)$$

Efecto en la softmax sin escalamiento: Para dimensiones habituales ($d_k = 64$ o 128), el producto punto alcanza magnitudes de orden $\sqrt{d_k} \approx 8 - 11$. Al alimentar logits tan dispares a la función softmax ($S_i = e^{z_i} / \sum_j e^{z_j}$), la distribución se polariza hacia un vector casi *one-hot*. Dado que la derivada es $\frac{\partial S_i}{\partial z_j} = S_i(\delta_{ij} - S_j)$, cuando $S_i \rightarrow 1$ o $S_i \rightarrow 0$ el gradiente colapsa exponencialmente hacia cero, deteniendo el aprendizaje de los pesos por desvanecimiento del gradiente.

Multi-Head Attention (MHA): MHA proyecta las entradas a h subespacios independientes de menor dimensión $d_k = d_{\text{model}}/h$:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \quad (4)$$

Esto permite al modelo atender simultáneamente a múltiples aspectos complementarios (sintaxis local, correferencia pronominal, dependencias semánticas de largo alcance). El número de parámetros **no cambia**: las h cabezas con matrices de tamaño $d_{\text{model}} \times (d_{\text{model}}/h)$ suman $h \cdot \frac{d_{\text{model}}^2}{h} = d_{\text{model}}^2$ por proyección, idéntico a una sola proyección de dimensión completa.

Codificación Posicional: La auto-atención pura es una operación equivariante a permutaciones: permutar el orden de los tokens de entrada permuta exactamente las salidas sin registrar alteración de significado. Para codificar el orden secuencial se inyectan embeddings de posición:

- **Sinusoidal fija [3]:** Funciones seno y coseno de frecuencias geométricas ($PE_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d})$). Permiten modelar desplazamientos relativos mediante combinaciones lineales y extrapolar a longitudes mayores.
- **Embeddings aprendidos (BERT, GPT-2):** Matrices entrenables $L_{\text{máx}} \times d_{\text{model}}$. Son altamente expresivas pero incapaces de generalizar más allá de su longitud prefijada.
- **RoPE (Rotary Position Embedding) [4]:** Rota los vectores Q y K en el plano complejo con ángulos proporcionales a su posición: el producto $(\tilde{q}_m)^T \tilde{k}_n$ codifica naturalmente la distancia relativa $(n - m)$, conservando la norma euclidiana y facilitando extrapolar de contexto.

Bloque Transformer: Conexiones residuales, LayerNorm y MLP: Cada bloque integra MHA y una red feed-forward bi-lineal por posición ($\text{FFN}(x) = \text{máx}(0, xW_1 + b_1)W_2 + b_2$) con dimensión interna $4d_{\text{model}}$. En *Post-LN* [3], la normalización se sitúa tras la suma residual ($x_{l+1} = \text{LN}(x_l + \text{Sublayer}(x_l))$), lo que introduce gradientes inestables en las primeras capas y demanda fases estrictas de calentamiento (*warmup*). En cambio, *Pre-LN* aplica la normalización a la entrada de cada subcapa ($x_{l+1} = x_l + \text{Sublayer}(\text{LN}(x_l))$), preservando un canal residual sin distorsión que garantiza convergencia robusta sin *warmup* hipercalibrado.

Complejidad y alternativas eficientes: El producto QK^T requiere $O(n^2d)$ operaciones y $O(n^2)$ en memoria de activación por cabeza. Principales propuestas de mitigación:

- **FlashAttention [5]:** Calcula softmax en bloques sobre memoria SRAM rápida sin escribir la matriz $n \times n$ en memoria global (HBM). Es exacta (sin aproximación), pero requiere kernels específicos de bajo nivel.
- **Atención por ventanas / Sparse (Longformer):** Limita la atención a un entorno local de tamaño $w \ll n$, con costo $O(n \cdot w)$, sacrificando interacción global directa en una sola capa.
- **Atención lineal (Linformer, Performer):** Aplica descomposición de kernel para operar $Q(K^TV)$ en costo $O(nd^2)$, sacrificando la precisión de la distribución softmax pura.

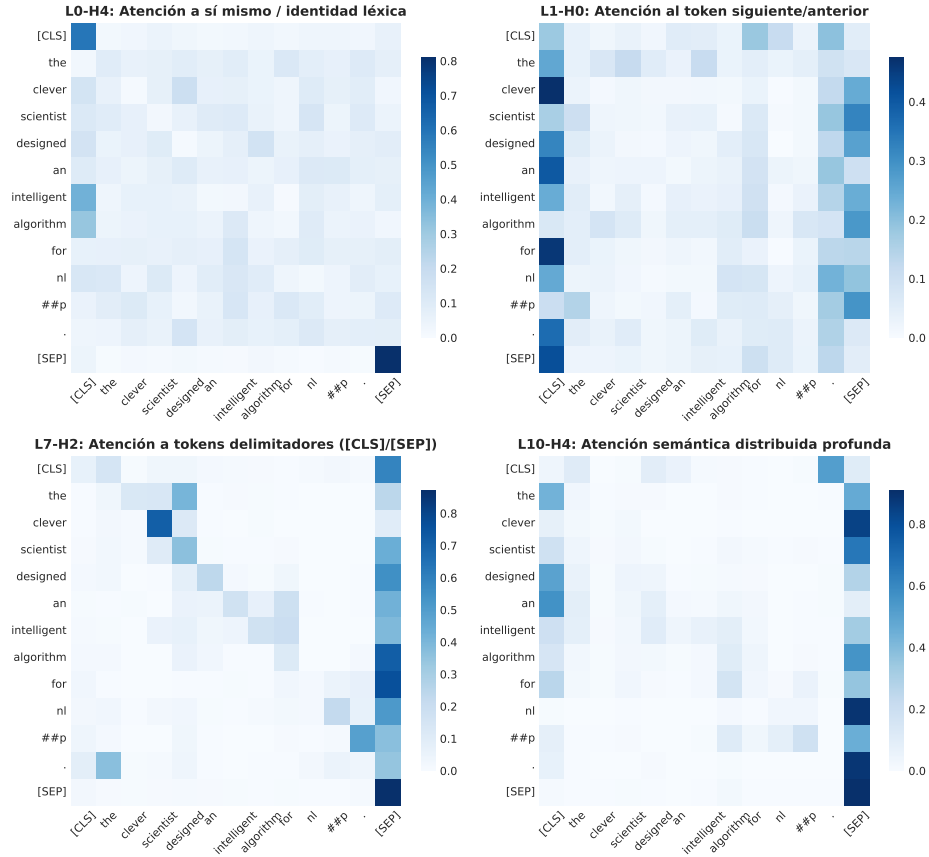


Figura 2: Mapas de calor en BERT mostrando cuatro patrones: identidad léxica (L0-H4), atención a token adyacente (L1-H0), concentración en delimitadores especiales [CLS] y [SEP] (L7-H2) y dispersión semántica contextual (L10-H4).

3. Atención en PyTorch y Hugging Face

- **F.scaled_dot_product_attention**: Función optimizada en PyTorch 2.x con despacho automático a FlashAttention o kernels nativos. Recibe `query`, `key`, `value`, y argumentos clave: `attn_mask` (máscara aditiva o booleana), `dropout_p`, `is_causal` (máscara causal rápida) y `scale` (factor de escala alternativo).
- **nn.MultiheadAttention**: Módulo de atención multi-cabeza (`embed_dim`, `num_heads`, `batch_first`, `kdim`, `vdim`). Su método `forward` admite `key_padding_mask` para omitir relleno (*padding*), `attn_mask`, `need_weights` y `average_attn_weights` (promedio o discriminación por cabeza).
- **nn.TransformerEncoderLayer**: Capa completa con `d_model`, `nhead`, `dim_feedforward`, `dropout` y `norm_first` (Pre-LN vs Post-LN). Con `generate_square_subsequent_mask(sz)` crea la máscara triangular causal rellena de $-\infty$.
- **Atención en Hugging Face**: Al invocar un modelo con `output_attentions=True`, la salida incluye `attentions`: tupla de L tensores (uno por capa) de dimensiones `[batch.size, num_heads, seq_len, seq_len]`.

4. Ejercicios Prácticos y Verificaciones Empíricas

Verificación numérica y ejercicio de escalamiento: Generamos 20 000 pares de vectores gaussianos $q, k \sim \mathcal{N}(0, I)$ para dimensiones $d_k \in \{16, 64, 128, 256, 512, 1024\}$. Como ilustra la Figura 1 (izquierda), las varianzas observadas fueron $[16.16, 64.67, 127.08, 252.39, 509.24, 1012.72]$, validando empíricamente que $\text{Var}(q \cdot k) = d_k$. Sin la división entre $\sqrt{d_k}$, la probabilidad máxima de la softmax satura rápidamente hacia 0.971, eliminando gradientes, mientras que con el factor escala se conserva equilibrada cerca de 0.125. Asimismo, la implementación manual desarrollada en el notebook produjo salidas numéricas idénticas a PyTorch ($< 10^{-6}$). En la Figura 1 (derecha) se constata el crecimiento de memoria (768 MB por capa a 4096 tokens) y FLOPs característicos de $O(n^2)$.

Mapas de calor en BERT: En la Figura 2 evaluamos la oración “*The clever scientist designed an intelligent algorithm for NLP.*” en BERT, identificando cuatro patrones canónicos:

1. *Atención a la propia palabra (L0-H4)*: Fuerte concentración diagonal que preserva los embeddings iniciales.
2. *Tokens adyacentes (L1-H0)*: Banda subdiagonal orientada al token contiguo (*clever* $\rightarrow$ *scientist*).
3. *Delimitadores especiales (L7-H2)*: Absorción en [CLS] y [SEP] actuando como comodines neutros.
4. *Semántica contextual profunda (L10-H4)*: Conexiones cruzadas y no locales entre términos afines.

Resolución de correferencias (Esquema de Winograd): Analizamos el par:

- S_1 : “*The animal didn’t cross the street because it was too tired.*”

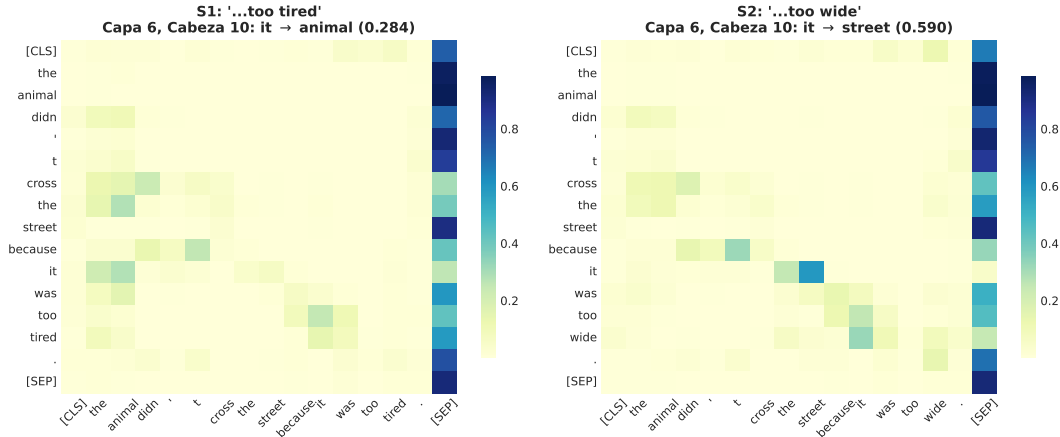
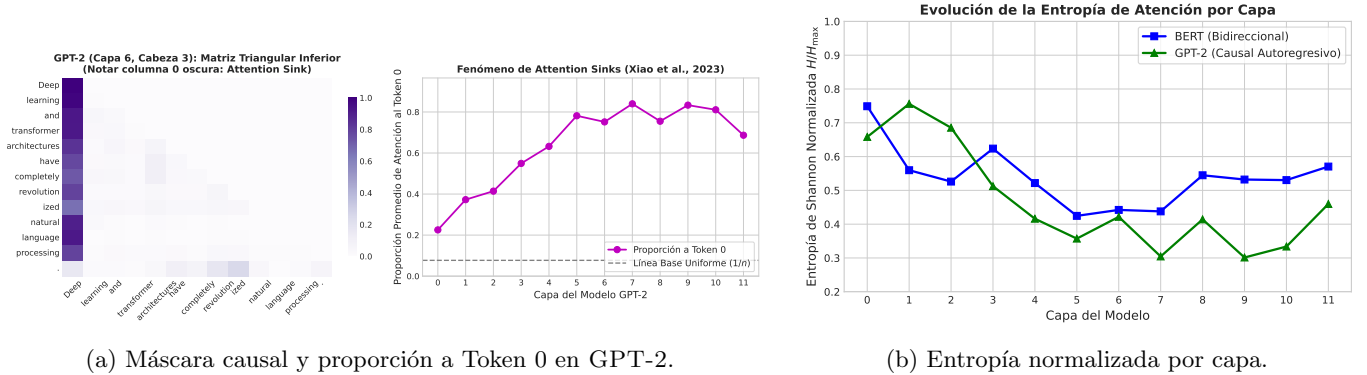


Figura 3: Resolución de correferencia en esquema Winograd con BERT (Capa 6, Cabeza 10). En S_1 (*too tired*), *it* atiende fuertemente a *animal* (0.284). En S_2 (*too wide*), la atención de *it* cambia radicalmente hacia *street* (0.590).



(a) Máscara causal y proporción a Token 0 en GPT-2.

(b) Entropía normalizada por capa.

Figura 4: (a) Causalidad triangular inferior y *attention sinks* en GPT-2. (b) Dinámica de entropía: las capas profundas se tornan más concentradas (menor entropía).

■ S_2 : “The animal didn’t cross the street because it was too wide.”

Exploramos las 144 cabezas de BERT buscando maximizar el contraste de atención de “it” hacia “animal” en S_1 y hacia “street” en S_2 . La **Capa 6, Cabeza 10** arrojó la máxima desambiguación ($\Delta = 0.7954$): en S_1 asigna 0.2843 a *animal* y solo 0.0671 a *street*; en S_2 asigna 0.5902 a *street* y 0.0119 a *animal* (Figura 3), comprobando que el modelo resuelve correferencias sintáctico-semánticas en capas intermedias.

Atención causal y Attention Sinks en GPT-2: Verificamos computacionalmente que la matriz de atención de GPT-2 es estrictamente triangular inferior (valores sobre la diagonal idénticamente nulos). Al estudiar la distribución de pesos, confirmamos de forma rotunda el fenómeno de **Attention Sinks** [6]: el **token 0** recibe en promedio el 22.5 % de atención en la capa 0, pero a partir de la capa 4 absorbe entre el 75 % y el 84 % de la atención de los tokens posteriores (Figura 4a). Esto ocurre porque la softmax impone que las sumas de fila sean 1; al no requerir atención sustantiva en ciertos pasos, el modelo vierte el exceso numérico en el primer token disponible.

Entropía de atención por capa (BERT vs. GPT-2): Calculamos la entropía de Shannon normalizada $H/H_{\max}$ (Figura 4b). En ambos modelos, las capas iniciales exhiben entropía elevada (0.75–0.76), correspondiente a distribuciones dispersas. En capas intermedias y profundas, la entropía decae marcadamente (0.42 en BERT y 0.30 en GPT-2), reflejando una focalización especializada en dependencias concretas.

5. Discusión y Análisis Crítico

1. Ventajas de escalamiento frente a RNNs: El entrenamiento de RNNs estaba encadenado al paso temporal $h_t = f(h_{t-1}, x_t)$, imposibilitando paralelizar la secuencia en GPUs. Además, el camino informacional $O(n)$ degradaba los gradientes. La auto-atención sustituyó la recurrencia por álgebra lineal paralelizable masiva en un paso constante $O(1)$. Pese al costo cuadrático, la regularidad del flujo computacional permitió expandir los modelos hacia las leyes de escala de Chinchilla/Kaplan.

2. Especialización y redundancia de cabezas: Aunque identificamos cabezas altamente especializadas (ej. correferencia en L6-H10), trabajos canónicos de poda (*pruning*) [7] demuestran que una porción notable de cabezas puede descartarse durante la inferencia sin merma de desempeño; el grueso del trabajo crítico recae sobre un núcleo reducido de cabezas.

3. ¿Los pesos de atención son explicación causal? Jain y Wallace [8] mostraron que distribuciones de atención

completamente diferentes pueden generar idéntica predicción, arguyendo que “atención no es explicación”. Wiegrefe y Pinter [9] precisaron que la atención refleja el flujo intermedio de información, pero debido a la no linealidad de las FFN subsiguientes, los pesos de atención describen correlaciones de representación y no causalidad estricta.

4. Cuándo preferir una RNN o CNN sobre un Transformer: Las RNNs (y arquitecturas afines como Mamba o RWKV) siguen siendo preferibles en dispositivos con memoria RAM ultrarreducida (Edge/TinyML) gracias a su inferencia con memoria constante $O(1)$, evitando el costo del KV cache. Por su parte, las CNNs ofrecen un sesgo inductivo traslacional insustituible para visión o audio crudo de bajo costo paramétrico.

5. Procesamiento de secuencias de 100,000 tokens: Una secuencia de 10^5 tokens colapsa la memoria convencional (requiriendo cientos de gigabytes solo para matrices de atención). Las soluciones estándar incluyen: FlashAttention-2 acoplado a RingAttention (paralelismo entre GPUs), ventanas de atención deslizante con *attention sinks* fijos (StreamingLLM), modelos de estado en el espacio (Mamba) con memoria $O(1)$ en generación, y arquitecturas de recuperación aumentada (RAG).

6. Conclusiones

El mecanismo de atención redefinió el procesamiento de secuencias eliminando el cuello de botella recurrente. Verificamos que el factor $1/\sqrt{d_k}$ estabiliza la varianza unitaria de los logits e impide el colapso de gradientes en la softmax. Los análisis empíricos confirman la especialización de cabezas en BERT y la naturaleza de sumidero numérico del token inicial en GPT-2. La combinación de estos principios sustenta los avances contemporáneos del Deep Learning.

Repositorio Git del proyecto: <https://github.com/Javier-Espana/HT2-DL.git>

Referencias

- [1] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *arXiv preprint arXiv:1409.0473*, 2014.
- [2] M.-T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” *Empirical Methods in Natural Language Processing (EMNLP)*, 2015.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [4] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with IO-awareness,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [6] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023.
- [7] E. Voita, D. Talbot, F. Goncarenco, R. Sennrich, and I. Titov, “Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned,” in *Association for Computational Linguistics (ACL)*, 2019, pp. 5797–5808.
- [8] S. Jain and B. C. Wallace, “Attention is not explanation,” in *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019, pp. 94–106.
- [9] S. Wiegrefe and Y. Pinter, “Attention is not not explanation,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 11–20.